Project Proposal Report

Design and Implementation of EngBang v2

A Bangla-inspired Programming Language with a Modular Compiler Pipeline

Submitted by	Salek Bin Hossain
Roll / Lab Group	2107026 / A1
Department	Computer Science and Engineering
Institution	Khulna University of Engineering & Technology

Merged and corrected from the earlier proposal and the later EngBang v2 specification draft.

1. Objectives

- Design a distinctive programming language named EngBang using Bangla-influenced keywords instead of conventional C/C++ syntax.
- Define strict rules for variable declaration, updates, constants, default initialization, explicit type casting, and arithmetic expressions.
- Specify a binding-first conditional model with grouped AND/OR conditions and support for Python-like chained comparisons.
- Support bracket-based loops and typed user-defined functions with return statements and forward function calls.
- Implement the language through a modular compiler pipeline using Flex, Bison, C, AST construction, semantic analysis, TAC generation, and execution.
- Produce a clear, internally consistent proposal/report aligned with the current EngBang implementation.

2. Introduction

EngBang v2 is a custom programming language project developed for Compiler Lab. Its main goal is to create a syntax that is clearly different from traditional C, C++, or Python while still remaining precise, readable, and easy to parse. Instead of English-heavy keywords, EngBang uses Bangla-inspired words such as rakho, bodlao, bolo, banaw, pakka, khaali, barbe, komabe, and ferao.

A core design choice in EngBang is the binding-first model. Rather than placing comparison operators directly inside an if-style expression, EngBang first binds a variable to a comparison target using the form var -> value and then expresses the actual condition inside structured items such as {==}, {>=}, or {% 2}. This makes the language visually distinct while keeping the grammar deterministic.

The current proposal extends the earlier variables-and-conditionals draft into a complete project description. It now covers expressions, loops, functions, semantic rules, intermediate-code generation, execution, build workflow, and testing strategy.

3. Compiler Workflow Overview

The EngBang compiler follows a modular front-end and multi-pass execution pipeline:

Stage	Primary file(s)	Main responsibility
Lexical analysis	lexer.l	Convert raw source code into tokens.
Parsing	parser.y	Validate grammar and build the Abstract Syntax Tree (AST).
AST management	ast.h / ast.c	Represent the program structure in node form.
Semantic analysis	semantic.c, symtab.c, functab.c	Validate meaning: types, declarations, function signatures, loop rules, return rules.

TAC generation	tac.c	Generate three-address code from the validated AST.
Execution	exec.c, main.c	Interpret the program and produce the runtime output.

```
Source Code -> Lexer -> Parser -> AST -> Semantic Check -> TAC -> Interpreter -> Program
Output
```

4. Motivation and Design Goals

- To design a language that feels original and locally flavored instead of copying mainstream syntax.
- To keep the grammar parser-friendly by using explicit separators, strict termination, and highly regular statement shapes.
- To separate language phases into understandable modules so that compiler construction can be studied step by step.
- To support enough constructs for meaningful programs: variables, arithmetic, conditions, loops, and functions.
- To make semantic errors explicit, such as redeclaration, undeclared use, type mismatch, invalid cast, wrong return type, and invalid loop control.

5. Language Specification

5.1 Data Types

Type	Meaning	Valid values / examples
shonkha	Integer	-5, 0, 10, 999
doshomik	Float	3.14, -0.50, 10.0
lekha	Text / String	"hello", "Talha \"Safin\""
shotto	Boolean	0 or 1 only

Boolean convention: shotto values are represented strictly as 0 (false) and 1 (true).

5.2 Keywords, Identifiers, and Statement Termination

Identifiers must start with a letter (A-Z or a-z) or underscore (_). Digits are allowed after the first character. Reserved keywords cannot be used as identifiers.

Category	Items
Core keywords	rakho, bodlao, pakka, khaali, ar, bolo, banaw
Control / function keywords	barbe, komabe, thamo, ferao
Type keywords	shonkha, doshomik, lekha, shotto
Structural tokens	->, |, [ ], { }, commas, parentheses, braces

Regular statements end with the pipe symbol |. Loop blocks and function definitions end at their closing brace and do not take an extra trailing |. A full conditional chain ends with a single trailing | after the final branch.

5.3 Declaration, Default Values, Updates, and Constants

Declaration syntax:

```
value rakho var_name -> type |
```

Default initialization syntax:

```
khaali rakho var_name -> type |
```

Update syntax:

```
value bodlao var_name |
```

Constant declaration syntax:

```
value pakka rakho NAME -> type |
```

- rakho declares a new variable and initializes it.
- Redeclaration of the same variable name is a semantic error.
- khaali assigns the language-defined default value for the declared type.
- bodlao updates an already-declared variable and must preserve type compatibility.
- pakka variables are immutable after declaration.

Type	Default value with khaali
shonkha	0

doshomik	0.0
lekha	""
shotto	0

Example:

```
10 rakho x -> shonkha |
khaali rakho title -> lekha |
10 pakka rakho LIMIT -> shonkha |
20 bodlao x |
```

5.4 Explicit Type Casting (banaw)

EngBang keeps typing strict. When an explicit conversion is intended, the banaw keyword must be used.

```
value banaw castType rakho var_name -> type |
value banaw castType bodlao var_name |
```

Examples:

```
"123" banaw shonkha rakho n -> shonkha |
"3.50" banaw doshomik rakho f -> doshomik |
123 banaw lekha rakho s -> lekha |
"999" banaw shonkha bodlao n |
```

5.5 Arithmetic Expressions

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division

Operator precedence follows normal mathematics: * and / bind more tightly than + and -. Parentheses may be used to override precedence.

- shonkha OP shonkha produces shonkha.
- If either operand is doshomik, the result is doshomik.

- Arithmetic on lekha or shotto operands is a semantic error.

Example:

```
10 rakho a -> shonkha |
3 rakho b -> shonkha |
bolo a + b |
bolo (a + b) * 2 |
```

5.6 Conditional System

EngBang conditionals follow a binding-first model with grouped conditions.

```
# IF
<bindings> {
  <groups> {
    <statements>
  }
}

# ELSEIF
<bindings> {
  <groups> {
    <statements>
  }
}

# ELSE
[] {
  <statements>
}
|
```

- A binding has the form var -> value and bindings are chained with ar.
- A single [...] group means AND across all bindings.
- Multiple groups written consecutively, such as [...][...], represent OR.
- Each condition group must contain exactly as many items as there are bindings.
- {} means skip this binding in the current group.
- [] is reserved exclusively for the final else branch.

Condition kind	Examples	Meaning
Comparator-only	{==}, {!=}, {>=}, {<=}, {>}, {<}	Compare the bound variable directly against the bound value.
Transform then compare	{% 2}, {+ 10}, {>> 1}, {^ 5}	Apply the transform first, then test equality with the bound value.
Skip item	{}	Ignore this binding in the current AND group.

Example:

```
cgpa -> 3.75 ar attendance -> 80 ar hasQuota -> 1 {
  [{>=}, {>=}, {}][{}, {}, {==}] {
    bolo "Eligible" |
  }
}
[] { bolo "Not eligible" | }
|
```

The same variable may appear more than once in a binding list, which makes Python-like chained comparisons possible.

5.7 Loops

EngBang uses a bracket-based loop header rather than C-style for or while syntax.

```
[varName -> startVal, varName -> endVal] {
  <body statements>
  varName barbe
}

[varName -> startVal -> type, varName -> endVal] {
  <body statements>
  varName komabe
}
```

- The variable name in the end position must match the variable named in the start position.
- barbe means increment after each iteration; komabe means decrement; thamo executes once and then stops.
- The control keyword is the final item inside the loop block and does not end with |.
- The controlling values must be type-compatible with the loop variable.

Example:

```
1 rakho i -> shonkha |
[i -> 1, i -> 5] {
  bolo i |
  i barbe
}
```

5.8 Functions and Return Statements

EngBang supports typed user-defined functions, typed parameters, return statements, and forward calls.

```
funcName(param1 -> type, param2 -> type, ...) -> returnType {
  <body statements>
  ferao expr |
}
```

- Parameters are typed and separated by commas.
- The return type is declared after the closing parenthesis using -> returnType.
- Function calls may appear wherever an expression is valid.

- Forward calls are supported through semantic pre-registration of function signatures.
- The type of the expression returned by ferao must match the declared return type.

Example:

```
bolo add(10, 5) |

add(x -> shonkha, y -> shonkha) -> shonkha {
  ferao x + y |
}
```

5.9 C to EngBang Example Mapping

A few compact comparisons help demonstrate that EngBang expresses familiar logic in its own syntax.

```
C:
if (x % 2 == 0) printf("jor\n");
else printf("bijor\n");

EngBang:
x -> 0 {
  [{% 2}] { bolo "jor" | }
}
[] { bolo "bijor" | }
|
```

```
C:
for (int i = 1; i <= 5; i++) printf("%d\n", i);

EngBang:
1 rakho i -> shonkha |
[i -> 1, i -> 5] {
  bolo i |
  i barbe
}
```

6. Compiler Architecture and Project Structure

The proposal is aligned with the current modular EngBang implementation. The project is structured as follows:

```
projectr/
├── include/
│   ├── value.h
│   ├── ast.h
│   ├── symtab.h
│   ├── functab.h
│   ├── semantic.h
│   └── tac.h
├── src/
│   ├── main.c
│   ├── lexer.l
│   ├── parser.y
│   ├── ast.c
```

```
    │   ├── value.c
    │   ├── symtab.c
    │   ├── functab.c
    │   ├── semantic.c
    │   ├── tac.c
    │   └── exec.c
    ├── tests/
    ├── Makefile
    └── README.md
```

Pass	Primary file(s)	Purpose	Must not do
Parse	lexer.l + parser.y	Create tokens and build AST nodes.	Directly update symbol tables or execute the program.
Semantic	semantic.c	Validate types, declarations, loop/function consistency, and populate tables.	Generate TAC or execute statements.
TAC	tac.c	Generate three-address code from the validated AST.	Change semantic meaning or interpret execution.
Execution	exec.c + main.c	Run the program after previous stages succeed.	Perform semantic validation.

7. Build, Run, and Error Reporting

Recommended build methods:

```
# Using Make
mingw32-make

# Manual build
bison -d -o src/parser.tab.c src/parser.y
flex -o src/lex.yy.c src/lexer.l
gcc -I include -Wall -Wextra -Wno-unused-function -o compiler.exe ^
    src/main.c src/ast.c src/value.c src/symtab.c src/functab.c ^
    src/semantic.c src/tac.c src/exec.c src/parser.tab.c src/lex.yy.c
```

Sample run:

```
Get-Content .\tests\test_01_variables.txt | .\compiler.exe
```

Typical output pattern:

```
TAC: ...
TAC: ...
```

```
    <actual program output>
    Parsing + Semantic OK.
```

The compiler reports parse errors and semantic errors separately. Examples include missing statement terminators, undeclared variables, type mismatch, invalid updates of pakka variables, wrong function return types, and inconsistent loop headers.

8. Testing and Validation Plan

Validation is based on both positive and negative test cases.

- Positive tests verify correct execution of variables, constants, casts, arithmetic, conditionals, loops, functions, and forward calls.
- Negative tests verify error handling for undeclared variables, redeclaration, type mismatch, invalid casts, constant updates, return-type mismatch, loop-variable mismatch, and syntax mistakes.
- A combined integration test exercises multiple language constructs together to confirm that the compiler pipeline remains consistent end to end.

Test category	What it proves
Valid program tests	The compiler accepts correct EngBang source, generates TAC, and produces the expected runtime output.
Semantic error tests	The compiler rejects meaning-level mistakes after parsing.
Syntax error tests	The parser rejects malformed token sequences before semantic analysis.
Combined integration test	Variables, loops, functions, and conditions work together in one program.

9. Expected Outcome

- A complete language specification for EngBang v2.
- A working compiler/interpreter built with Flex, Bison, and C.
- A modular source tree with separate files for AST, semantics, tables, TAC, and execution.
- A test suite that demonstrates both successful execution and proper error detection.
- A language that is visibly different from C/C++ syntax while remaining formally specified and implementable.

10. Limitations and Future Work

- The current scope focuses on core programming constructs rather than large-scale software features.

- Possible future extensions include arrays, richer string operations, user input, file handling, and more advanced libraries.
- A future version may also target machine code or bytecode rather than using interpretation after TAC.

11. Conclusion

This final corrected and merged proposal presents EngBang as both a language-design project and a compiler-construction project. It integrates the earlier corrected report with the useful additions from the later EngBang v2 draft: clearer loop and function specifications, explicit statement-termination rules, a cleaner compiler architecture section, build instructions, testing strategy, and a complete keyword overview. As a result, the document now reflects the full implemented scope of the EngBang project in a more complete and submission-ready form.